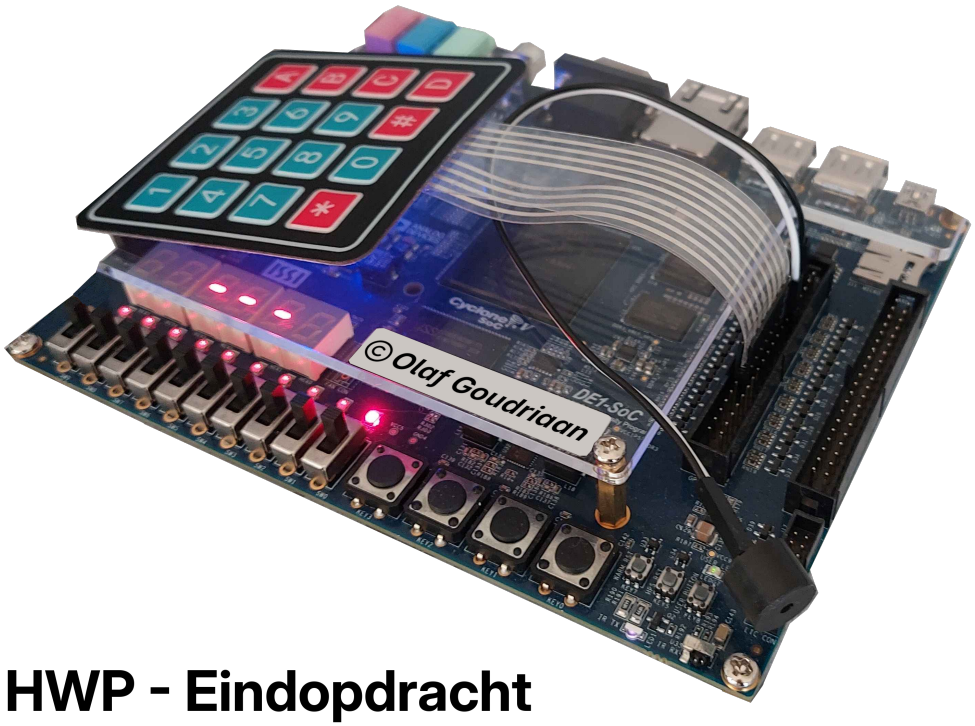


HWP - Eindopdracht

Minor elektrotechniek

Eindopdracht HWP
Embedded Systems

Olaf Goudriaan, 1071349
Fanuel Hadgu, 1087006
2026-2027
Broeders, J.Z.M. (Harry)
ELEHWP10

Inhoudsopgave

1 Werkend prototype **2**

2 Opdracht **3**
2.1 Statemachine . . . 3
2.2 Keypad . . . 3
2.3 Passcode . . . 3
2.4 Buzzer . . . 3
2.5 Sevensegment Driver . . . 3
2.6 Counter . . . 3

3 Ontwerpen **3**

4 Testbenches **9**
4.1 Buzzer . . . 9
4.2 Clock Divider . . . 13
4.3 Keypad . . . 16
4.4 Passcode . . . 19
4.5 Seven Segment Driver . . . 23
4.6 State Machine . . . 25

5 VHDL Code **32**
5.1 Keypad . . . 36
5.2 Passcode . . . 40
5.3 Buzzer . . . 43
5.4 Sevensegment driver . . . 46
5.5 Statemachine . . . 47
5.6 Clockdivider . . . 50

1 Werkend prototype

Een youtube link naar het werkende prototype: https://www.youtube.com/shorts/qaX7Bm6lbgI

2 Opdracht

In deze opdracht gaat de HWP opdracht van de keypad uitgebreid worden tot een deurslot met keypad. Ook zal er een buzzer aan toegevoegd worden.

2.1 Statemachine

In het ontwerp zal een statemachine te vinden zijn met vier states. Als het systeem opstart of reset zal deze naar de 'closed' state gaan. Wanneer iemand begint met cijfers invoeren zal de state veranderen naar "progress". Vanuit "progress" kan de state drie kanten op gaan. Met '*' zal de state resetten naar 'closed' en een reset geluidje geven, als de code van 4 cijfers fout is zal de buzzer een fout geluidje geven (signaal is te bepalen), als de code juist is zal er een 'juist signaal' klinken (signaal is te bepalen). Ook zal de led die bij alle andere states aan staat, uit gaan. Na een aantal seconden zal de state terug naar 'closed' gaan. Wanneer de state terug naar closed gaat vanuit fout of correct zal er een geluidje klinken.

Als er een cijfer ingevoerd wordt, zal er een geluidje klinken. En op de sevensegment displayes zal een '-' komen op de plek van het getal.

2.2 Keypad

Voor dit component moet een module gemaakt worden dat controleerd of er een key is ingedrukt en de key doorgeeft als output. Daarnaast geeft deze ook een puls als er een cijfer is ingedrukt. Ook geeft deze de keypad reset signaal, keypad done en pressed signaal.

2.3 Passcode

Dit is een component dat de keys opslaat wanneer deze zijn ingedrukt en daarna controleerd of de cijfercode correct is.

2.4 Buzzer

Dit is een component dat de buzzer aanstuurt om de geluiden te geven. Je kunt selecteren welk geluid door het geluid te selecteren. De geluiden zijn: 'key pressed', 'code fault', 'code correct' en 'reset'. De exacte geluiden voor deze signalen moeten nog bepaald worden.

2.5 Sevensegment Driver

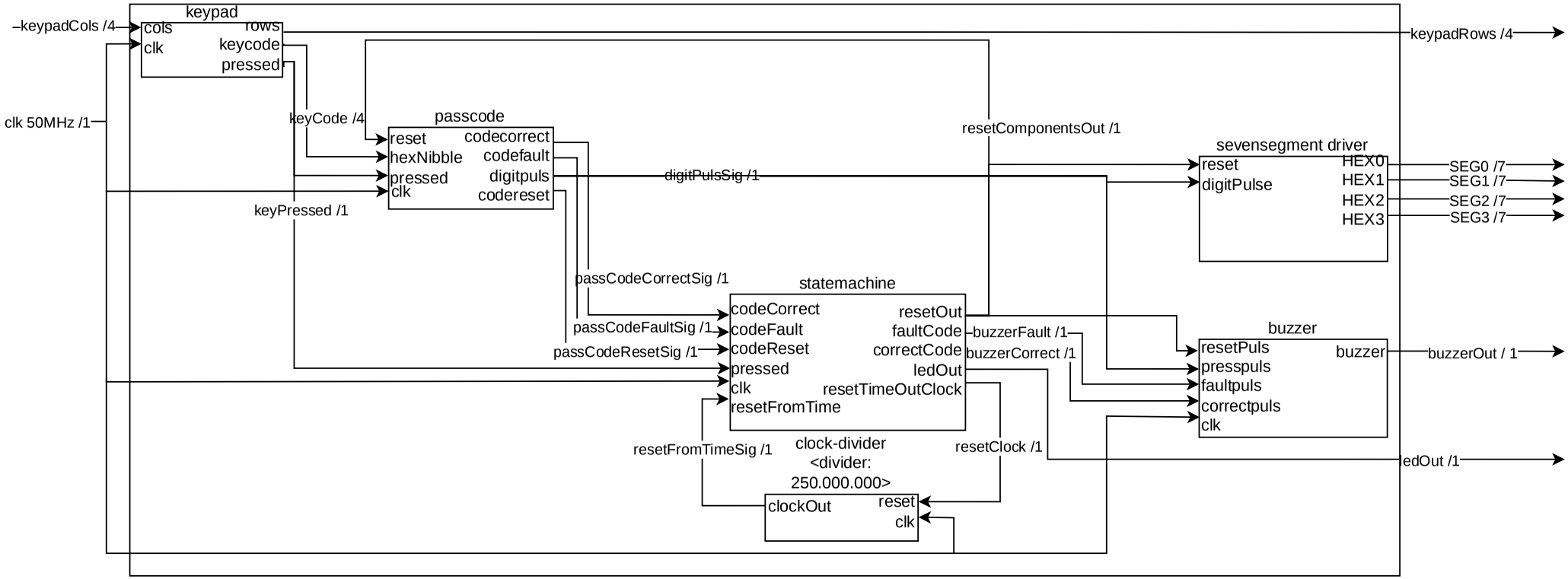
Deze driver moet de sevensegments aansturen. En bij iedere cijferspuls zal deze een extra '-' op een display zetten.

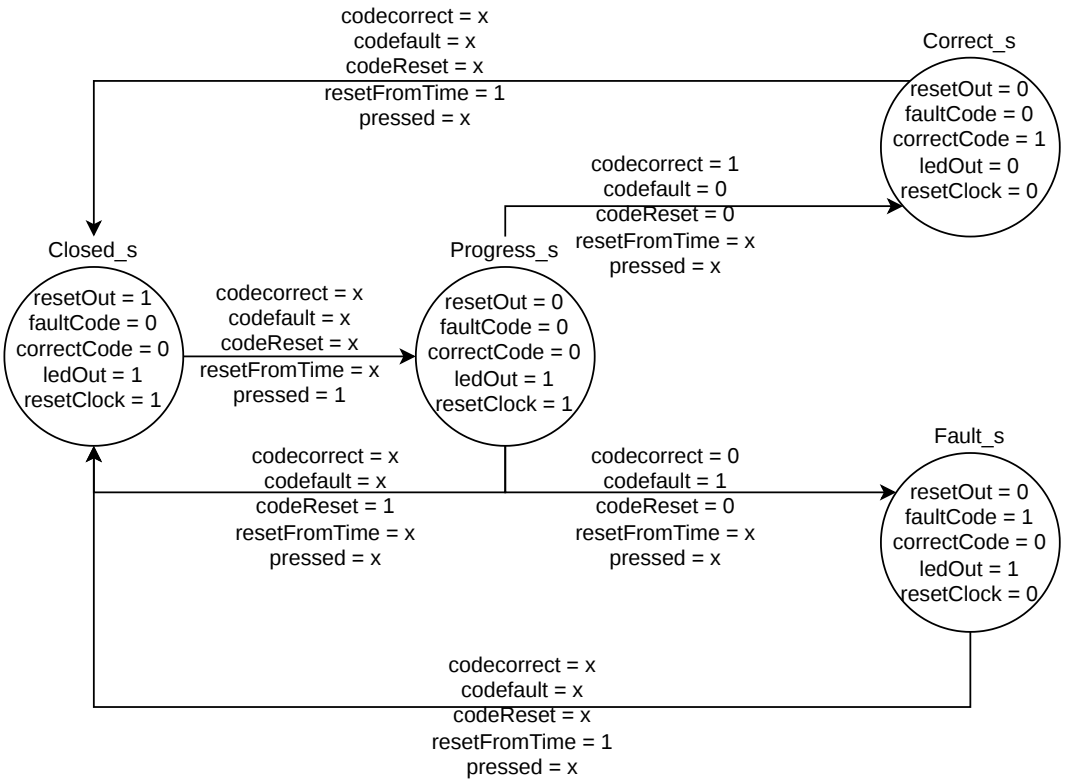
2.6 Counter

Om terug te gaan naar de closed state wordt een counter gebruikt.

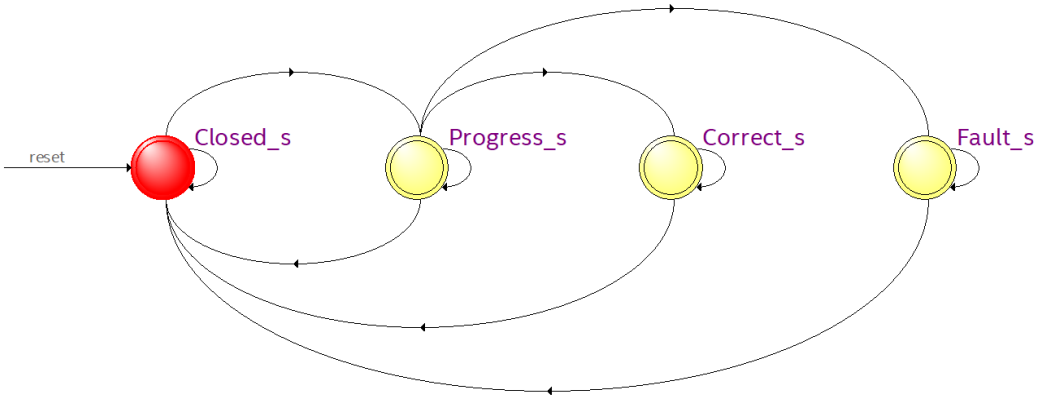
3 Ontwerpen

Hieronder zijn de hoofd diagrammen te zien van het systeem. Dit zijn het toetstandsdiagram en het blokdiagram.

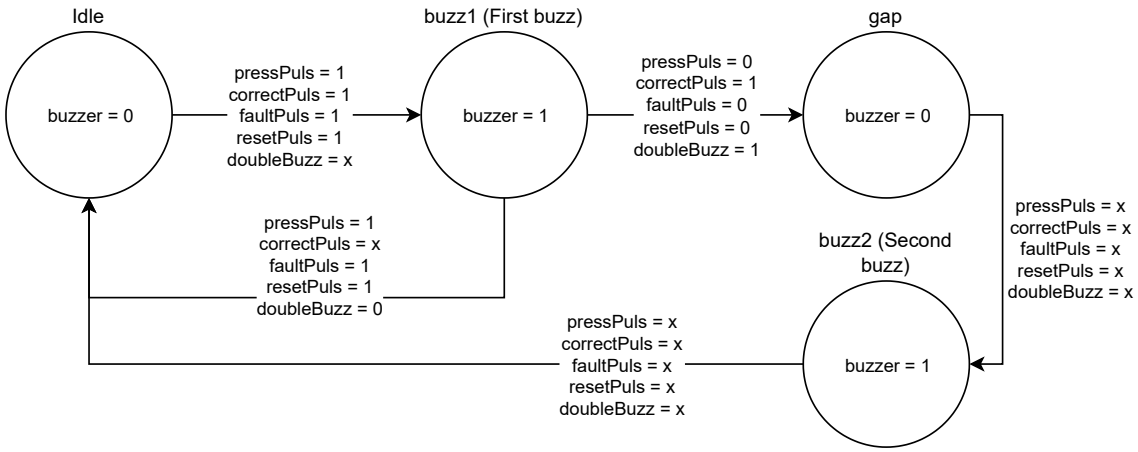

keypad
cols
clk
rows
keycode
pressed
keypadCols /4
keypadRows /4
clk 50MHz /1
keyCode /4
keyPressed /1
passcode
reset
hexNibble
pressed
clk
codecorrect
codefault
digitpuls
codereset
resetComponentsOut /1
digitPulsSig /1
sevensegment driver
reset
digitPulse
HEX0
HEX1
HEX2
HEX3
SEG0 /7
SEG1 /7
SEG2 /7
SEG3 /7
passCodeCorrectSig /1
passCodeFaultSig /1
passCodeResetSig /1
statemachine
codeCorrect
codeFault
codeReset
pressed
clk
resetFromTime
resetOut
faultCode
correctCode
ledOut
resetTimeOutClock
buzzerFault /1
buzzerCorrect /1
buzzer
resetPuls
presspuls
faultpuls
correctpuls
clk
buzzer
buzzerOut / 1
ledOut /1
resetFromTimeSig /1
clock-divider
<divider:
250.000.000>
clockOut
reset
clk
resetClock /1

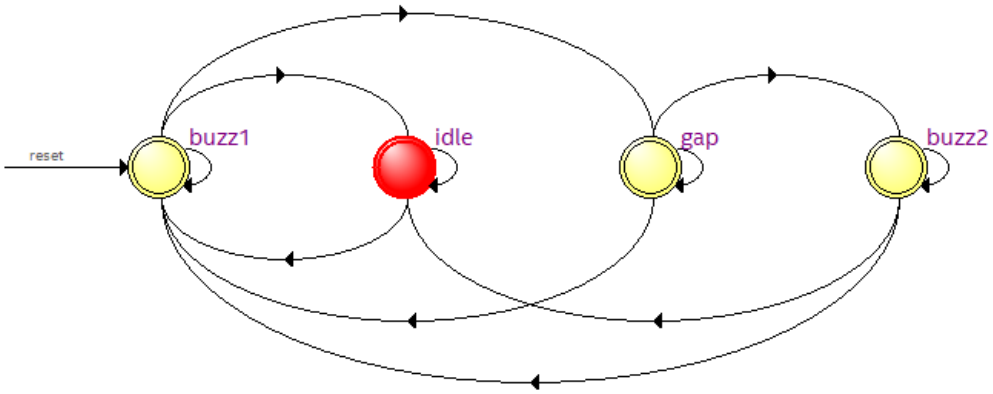

codecorrect = x
codefault = x
codeReset = x
resetFromTime = 1
pressed = x
Correct_s
resetOut = 0
faultCode = 0
correctCode = 1
ledOut = 0
resetClock = 0
codecorrect = 1
codefault = 0
codeReset = 0
resetFromTime = x
pressed = x
Closed_s
resetOut = 1
faultCode = 0
correctCode = 0
ledOut = 1
resetClock = 1
codecorrect = x
codefault = x
codeReset = x
resetFromTime = x
pressed = 1
Progress_s
resetOut = 0
faultCode = 0
correctCode = 0
ledOut = 1
resetClock = 1
Fault_s
resetOut = 0
faultCode = 1
correctCode = 0
ledOut = 1
resetClock = 0
codecorrect = x
codefault = x
codeReset = 1
resetFromTime = x
pressed = x
codecorrect = 0
codefault = 1
codeReset = 0
resetFromTime = x
pressed = x
codecorrect = x
codefault = x
codeReset = x
resetFromTime = 1
pressed = x

De statemachine is vergeleken met de statemachine vanuit quartus software. Deze is identiek aan de statemachine die vooraf was ontworpen. Deze is hieronder te zien.



Daarnaast is voor de buzzer ook nog een toestandsdiagram gemaakt, Deze is hieronder te vinden.


Idle
buzzer = 0
pressPuls = 1
correctPuls = 1
faultPuls = 1
resetPuls = 1
doubleBuzz = x
buzz1 (First buzz)
buzzer = 1
pressPuls = 0
correctPuls = 1
faultPuls = 0
resetPuls = 0
doubleBuzz = 1
gap
buzzer = 0
pressPuls = x
correctPuls = x
faultPuls = x
resetPuls = x
doubleBuzz = x
buzz2 (Second buzz)
buzzer = 1
pressPuls = x
correctPuls = x
faultPuls = x
resetPuls = x
doubleBuzz = x
pressPuls = 1
correctPuls = x
faultPuls = 1
resetPuls = 1
doubleBuzz = 0


reset
buzz1
idle
gap
buzz2

4 Testbenches

Hieronder zijn de testbenches te vinden. De testbenches zijn gegenereerd met AI.

4.1 Buzzer

De testbench voert hier de volgende tests uit:
1. Er wordt gekeken of er de buzzer aan gaat zonder invoer. Dit slaagt als de buzzer niet aangaat.
2 t/m 5. De bestemde functies worden getest: 1 korte piep voor invoer, twee korte piepjes voor een correct wachtwoord, en een lange piep als de code fout is, of als de reset-knop wordt ingedrukt.
6. Er wordt getest of de buzzer langer aangaat dan zou moeten wanneer een knop lang ingedrukt blijft. Test slaagt als de buzzer uitgaat na de aangegeven lengte.
7. De prioriteit van de pulses wordt getest door alle pulsen hoog te maken. De prioriteit moet correct > fout > reset > press zijn. Test slaagt als de buzzer ,bij meerdere pulses, de puls met de hoogste prioriteit volgt.

```
--------------------------------------------------------------------------------
-- buzzer_tb.vhd
--
-- Testbench for the buzzer component (generic: clkFreq; ports: clk,
-- pressPuls, faultPuls, correctPuls, resetPuls, buzzer).
--
-- clkFreq is overridden with 800 so the beeps are short in simulation.
-- The 1/8, 3/8 and 1/2 second durations then become:
--   short beep = 100 clock cycles, gap = 300 cycles, long beep = 400 cycles
-- (with a 10 ns clock that is 1 us, 3 us and 4 us).
--
-- Checks:
--   1. No sound while idle.
--   2. pressPuls    -> one short beep.
--   3. correctPuls  -> two short beeps with a gap between them.
--   4. faultPuls    -> one long beep.
--   5. resetPuls    -> one long beep.
--   6. A signal held high only triggers one beep (edge detection).
--   7. Simultaneous pulses follow the priority correct > fault >
--      reset code > press.
--   8. A new pulse during a beep restarts the buzzer with the new pattern.
--------------------------------------------------------------------------------

library ieee;
use ieee.std_logic_1164.all;

entity buzzer_tb is
end entity;

architecture testbench of buzzer_tb is

  constant CLK_PERIOD : time    := 10 ns;
  constant CLK_FREQ   : natural := 800;

  constant SHORT_T : time := (CLK_FREQ / 8) * CLK_PERIOD;      -- 100 cycles
```

```
35    constant GAP_T   : time := (CLK_FREQ / 8) * 3 * CLK_PERIOD;  -- 300
       cycles
36    constant LONG_T  : time := (CLK_FREQ / 2) * CLK_PERIOD;      -- 400
       cycles
37
38    component buzzer is
39      generic (
40        clkFreq : natural := 50000000
41      );
42      port (
43        clk         : in  std_ulogic;
44        pressPuls   : in  std_ulogic;
45        faultPuls   : in  std_ulogic;
46        correctPuls : in  std_ulogic;
47        resetPuls   : in  std_ulogic;
48        buzzer      : out std_ulogic
49      );
50    end component;
51
52    signal clk         : std_ulogic := '0';
53    signal pressPuls   : std_ulogic := '0';
54    signal faultPuls   : std_ulogic := '0';
55    signal correctPuls : std_ulogic := '0';
56    signal resetPuls   : std_ulogic := '0';
57    signal buzzer_out  : std_ulogic;
58    signal finished    : boolean := false;
59
60  begin
61
62    duv : buzzer
63      generic map (
64        clkFreq => CLK_FREQ
65      )
66      port map (
67        clk         => clk,
68        pressPuls   => pressPuls,
69        faultPuls   => faultPuls,
70        correctPuls => correctPuls,
71        resetPuls   => resetPuls,
72        buzzer      => buzzer_out
73      );
74
75    -- Clock; stops when the test is finished so the simulation ends.
76    clk_process : process
77    begin
78      while not finished loop
79        clk <= '0';
80        wait for CLK_PERIOD / 2;
81        clk <= '1';
82        wait for CLK_PERIOD / 2;
83      end loop;
84      wait;
85    end process;
86
```

```
87  -- Watchdog: abort if the test hangs (e.g. an expected beep never comes)
     .
88  watchdog : process
89  begin
90    wait until finished for 2 ms;
91    assert finished report "Testbench timed out." severity failure;
92    wait;
93  end process;
94
95  stim_process : process
96
97    -- Give a one-clock-cycle pulse on the selected inputs, all at once.
98    procedure pulse (constant p, f, c, r : in boolean) is
99    begin
100     wait until rising_edge(clk);
101     wait for CLK_PERIOD / 4;
102     if p then pressPuls   <= '1'; end if;
103     if f then faultPuls   <= '1'; end if;
104     if c then correctPuls <= '1'; end if;
105     if r then resetPuls   <= '1'; end if;
106     wait for CLK_PERIOD;
107     pressPuls   <= '0';
108     faultPuls   <= '0';
109     correctPuls <= '0';
110     resetPuls   <= '0';
111   end procedure;
112
113   -- The buzzer must stay silent for the given time.
114   procedure check_silence (constant d : in time; constant tag : in
     string) is
115   begin
116     wait for d;
117     assert buzzer_out = '0' and buzzer_out'last_event >= d
118       report "test failed (" & tag & "): buzzer made an unexpected sound
     "
119       severity error;
120   end procedure;
121
122   -- Expect exactly one beep of length on_t, then silence.
123   procedure expect_single (constant on_t : in time; constant tag : in
     string) is
124     variable ta, tb : time;
125   begin
126     wait until rising_edge(buzzer_out);
127     ta := now;
128     wait until falling_edge(buzzer_out);
129     tb := now;
130     assert tb - ta = on_t
131       report "test failed (" & tag & "): beep lasted " & time'image(tb -
     ta) &
132         ", expected " & time'image(on_t)
133       severity error;
134     check_silence(GAP_T + SHORT_T + 5 * CLK_PERIOD, tag);
135   end procedure;
```

```
136
137     -- Expect two short beeps separated by the gap, then silence.
138     procedure expect_double (constant tag : in string) is
139       variable ta, tb, tc, td : time;
140     begin
141       wait until rising_edge(buzzer_out);
142       ta := now;
143       wait until falling_edge(buzzer_out);
144       tb := now;
145       wait until rising_edge(buzzer_out);
146       tc := now;
147       wait until falling_edge(buzzer_out);
148       td := now;
149       assert tb - ta = SHORT_T
150         report "test failed (" & tag & "): first beep lasted " & time'
    image(tb - ta)
151         severity error;
152       assert tc - tb = GAP_T
153         report "test failed (" & tag & "): gap lasted " & time'image(tc -
    tb)
154         severity error;
155       assert td - tc = SHORT_T
156         report "test failed (" & tag & "): second beep lasted " & time'
    image(td - tc)
157         severity error;
158       check_silence(GAP_T + SHORT_T + 5 * CLK_PERIOD, tag);
159     end procedure;
160
161     variable t1, t2 : time;
162   begin
163     report "Testing entity buzzer.";
164
165     ---------------------------------------------------------------
166     -- 1. Silence while idle (no reset input: the buzzer starts idle)
167     ---------------------------------------------------------------
168     check_silence(10 * CLK_PERIOD, "idle");
169
170     ---------------------------------------------------------------
171     -- 2. Key press: one short beep
172     ---------------------------------------------------------------
173     pulse(true, false, false, false);
174     expect_single(SHORT_T, "press");
175
176     ---------------------------------------------------------------
177     -- 3. Code correct: two short beeps
178     ---------------------------------------------------------------
179     pulse(false, false, true, false);
180     expect_double("code correct");
181
182     ---------------------------------------------------------------
183     -- 4. Code fault: one long beep
184     ---------------------------------------------------------------
185     pulse(false, true, false, false);
186     expect_single(LONG_T, "code fault");
```

```
187
188     ---------------------------------------------------------------
189     -- 5. Reset code: one long beep
190     ---------------------------------------------------------------
191     pulse(false, false, false, true);
192     expect_single(LONG_T, "reset code");
193
194     ---------------------------------------------------------------
195     -- 6. Input held high: only one beep (edge detection)
196     ---------------------------------------------------------------
197     pressPuls <= '1';
198     expect_single(SHORT_T, "press held high");
199     wait for 2 * SHORT_T;
200     assert buzzer_out = '0'
201       report "test failed: held press beeped again" severity error;
202     pressPuls <= '0';
203     check_silence(10 * CLK_PERIOD, "press released");
204
205     ---------------------------------------------------------------
206     -- 7. Simultaneous pulses: correct > fault > reset code > press
207     ---------------------------------------------------------------
208     pulse(true, true, true, true);
209     expect_double("all pulses: correct wins");
210
211     pulse(true, true, false, true);
212     expect_single(LONG_T, "press+fault+reset: long beep");
213
214     pulse(true, false, false, true);
215     expect_single(LONG_T, "press+reset: long beep");
216
217     report "Test completed.";
218     finished <= true;
219     wait;
220   end process;
221
222 end architecture;
223
224 -- Output:
225 -- # ** Note: Testing entity buzzer.
226 -- #    Time: 0 ps  Iteration: 0  Instance: /buzzer_tb
227 -- # ** Note: Test completed.
228 -- #    Time: 71125 ns  Iteration: 0  Instance: /buzzer_tb
```

Listing 1: buzzer_tb

4.2 Clock Divider

Er wordt gekeken of de clockOut-variabele hoog wordt na elke vier clock cycles. Dit is normaal elke 25 miljoen, maar om het testproces te versimpelen is een lager getal gekozen.

```
0 library ieee;
1 use ieee.std_logic_1164.all;
2
```

```
entity ClockDivider_tb is
end entity;

architecture simulation of ClockDivider_tb is

    -- Testbench signals
    signal reset    : std_ulogic := '0';
    signal clk      : std_ulogic := '0';
    signal clockOut : std_ulogic;

    -- Clock period
    constant CLK_PERIOD : time := 10 ns;

begin

    ------------------------------------------------------------------------
    -- Device Under Test
    ------------------------------------------------------------------------
    DUT : entity work.ClockDivider(behavioural)
        generic map (
            counter => 4
        )
        port map (
            reset    => reset,
            clockOut => clockOut,
            clk      => clk
        );


    ------------------------------------------------------------------------
    -- Generate the input clock
    ------------------------------------------------------------------------
    clk_process : process
    begin
        while true loop
            clk <= '0';
            wait for CLK_PERIOD / 2;

            clk <= '1';
            wait for CLK_PERIOD / 2;
        end loop;
    end process;


    ------------------------------------------------------------------------
    -- Test process
    ------------------------------------------------------------------------
    stim_proc : process
    begin

        ------------------------------------------------------------------
        -- Test reset
        ------------------------------------------------------------------
        reset <= '1';
```

```
57          wait for 15 ns;
58
59          assert clockOut = '0'
60              report "Test failed: clockOut should be 0 during reset"
61              severity error;
62
63          reset <= '0';
64
65
66          -------------------------------------------------------------
67          -- Wait for the first clock division cycle
68          --
69          -- With counter = 4, clockOut should toggle after 4 input
70          -- clock rising edges.
71          -------------------------------------------------------------
72          wait until rising_edge(clk);
73          wait until rising_edge(clk);
74          wait until rising_edge(clk);
75          wait until rising_edge(clk);
76
77          wait for 1 ns;
78
79          assert clockOut = '1'
80              report "Test failed: clockOut should toggle to 1"
81              severity error;
82
83
84          -------------------------------------------------------------
85          -- Wait for the second division cycle
86          -------------------------------------------------------------
87          wait until rising_edge(clk);
88          wait until rising_edge(clk);
89          wait until rising_edge(clk);
90          wait until rising_edge(clk);
91
92          wait for 1 ns;
93
94          assert clockOut = '0'
95              report "Test failed: clockOut should toggle back to 0"
96              severity error;
97
98
99          -------------------------------------------------------------
100         -- Test reset while the clock divider is running
101         -------------------------------------------------------------
102         reset <= '1';
103         wait for 1 ns;
104
105         assert clockOut = '0'
106             report "Test failed: reset should set clockOut to 0"
107             severity error;
108
109         reset <= '0';
110
```

```
111
112         report "All ClockDivider tests completed successfully!"
113             severity note;
114
115         wait;
116     end process;
117
118 end architecture;
119
120 -- Output:
121 -- # ** Note: All ClockDivider tests completed successfully!
122 -- #    Time: 87 ns  Iteration: 0  Instance: /clockdivider_tb
```

Listing 2: ClockDivider_tb

4.3 Keypad

Hier wordt getest of de invoer vanuit de keypad goed is vertaald naar een integer. Ook wordt de resetknop "*" getest.

```
0  library ieee;
1  use IEEE.numeric_std.all;
2  use IEEE.std_logic_1164.all;
3
4  --
       ----------------------------------------------------------------------------
5  -- Keypad Testbench Entity & Architecture
6  --
       ----------------------------------------------------------------------------
7  entity keypad_tb is
8  end entity;
9
10 architecture tb of keypad_tb is
11
12   -- Component Declaration for the Keypad under test
13   component keypad is
14     port(
15       cols    : in std_ulogic_vector(3 downto 0);
16       rows    : out std_ulogic_vector(3 downto 0);
17       clk     : in std_ulogic;
18       pressed : out std_ulogic;
19       keycode : out std_ulogic_vector(3 downto 0)
20     );
21   end component;
22
23   -- Testbench Signals
24   signal clk     : std_ulogic := '0';
25   signal cols    : std_ulogic_vector(3 downto 0) := "1111"; -- Default: no
       keys pressed (all high)
26   signal rows    : std_ulogic_vector(3 downto 0);
27   signal pressed : std_ulogic;
```

```
28    signal keycode : std_ulogic_vector(3 downto 0);
29
30    -- Simulation control signal to track test steps
31    signal test_step : integer := 0;
32
33    -- Clock period constant for 50 MHz (20 ns)
34    constant CLK_PERIOD : time := 20 ns;
35
36  begin
37
38    -- Instantiate the Unit Under Test (UUT)
39    uut: keypad
40      port map (
41        cols    => cols,
42        rows    => rows,
43        clk     => clk,
44        pressed => pressed,
45        keycode => keycode
46      );
47
48    -- 50 MHz Clock Generator Process
49    clk_process : process
50    begin
51      clk <= '0';
52      wait for CLK_PERIOD / 2;
53      clk <= '1';
54      wait for CLK_PERIOD / 2;
55    end process;
56
57    -- Stimulus & Dynamic Key Press Emulator Process
58    stimulus_process : process
59
60      -- Helper procedure to simulate pressing any key on the matrix
61      procedure press_key(
62        r_val     : std_ulogic_vector(3 downto 0);
63        c_val     : std_ulogic_vector(3 downto 0);
64        step_num  : integer;
65        key_desc  : string
66      ) is
67      begin
68        report "TEST " & integer'image(step_num) & ": Pressing key '" &
     key_desc & "'";
69        test_step <= step_num;
70        cols <= "1111"; -- Release before pressing next key
71
72        -- Wait until the scanner activates the target row
73        loop
74          wait until rising_edge(clk);
75          if rows = r_val then
76            cols <= c_val;
77            exit;
78          end if;
79        end loop;
80
```

```
81        -- Hold key long enough for sampling
82        wait for 1.5 ms;
83      end procedure;
84
85    begin
86      -- Initial state: No keys pressed
87      cols <= "1111";
88      wait for 2 ms;
89
90      --
        ------------------------------------------------------------------------
91      -- Standard 4x4 Keypad Layout Mapping:
92      -- Row 0 ("1110"): Col 0 ("1110") = '1', Col 1 ("1101") = '2', Col 2
        ("1011") = '3'
93      -- Row 1 ("1101"): Col 0 ("1110") = '4', Col 1 ("1101") = '5', Col 2
        ("1011") = '6'
94      -- Row 2 ("1011"): Col 0 ("1110") = '7', Col 1 ("1101") = '8', Col 2
        ("1011") = '9'
95      -- Row 3 ("0111"): Col 0 ("1110") = '*', Col 1 ("1101") = '0'
96      --
        ------------------------------------------------------------------------
97
98      -- Test Keys 1 to 9
99      press_key("1110", "1110",  1, "1");
100     press_key("1110", "1101",  2, "2");
101     press_key("1110", "1011",  3, "3");
102
103     press_key("1101", "1110",  4, "4");
104     press_key("1101", "1101",  5, "5");
105     press_key("1101", "1011",  6, "6");
106
107     press_key("1011", "1110",  7, "7");
108     press_key("1011", "1101",  8, "8");
109     press_key("1011", "1011",  9, "9");
110
111     -- Test Key '0' (Row 3, Col 1)
112     press_key("0111", "1101", 10, "0");
113
114     -- Test Key '*' (Row 3, Col 0 - active-low column "1110")
115     press_key("0111", "1110", 11, "*");
116
117     --
        ------------------------------------------------------------------------
118     -- Final Cleanup & End Simulation
119     --
        ------------------------------------------------------------------------
120     report "TEST 12: Releasing key and finishing simulation";
121     test_step <= 12;
122     cols <= "1111";
123     wait for 3 ms;
```

```
124
125     report "Simulation completed successfully!";
126     std.env.stop;
127   end process;
128
129 end architecture;
130
131 -- Output:
132 -- # ** Note: TEST 1: Pressing key '1'
133 -- #    Time: 2 ms  Iteration: 0  Instance: /keypad_tb
134 -- # ** Note: TEST 2: Pressing key '2'
135 -- #    Time: 4000010 ns  Iteration: 0  Instance: /keypad_tb
136 -- # ** Note: TEST 3: Pressing key '3'
137 -- #    Time: 5500010 ns  Iteration: 0  Instance: /keypad_tb
138 -- # ** Note: TEST 4: Pressing key '4'
139 -- #    Time: 7000010 ns  Iteration: 0  Instance: /keypad_tb
140 -- # ** Note: TEST 5: Pressing key '5'
141 -- #    Time: 9000010 ns  Iteration: 0  Instance: /keypad_tb
142 -- # ** Note: TEST 6: Pressing key '6'
143 -- #    Time: 10500010 ns  Iteration: 0  Instance: /keypad_tb
144 -- # ** Note: TEST 7: Pressing key '7'
145 -- #    Time: 12000010 ns  Iteration: 0  Instance: /keypad_tb
146 -- # ** Note: TEST 8: Pressing key '8'
147 -- #    Time: 14000010 ns  Iteration: 0  Instance: /keypad_tb
148 -- # ** Note: TEST 9: Pressing key '9'
149 -- #    Time: 15500010 ns  Iteration: 0  Instance: /keypad_tb
150 -- # ** Note: TEST 10: Pressing key '0'
151 -- #    Time: 17000010 ns  Iteration: 0  Instance: /keypad_tb
152 -- # ** Note: TEST 11: Pressing key '*'
153 -- #    Time: 19000010 ns  Iteration: 0  Instance: /keypad_tb
154 -- # ** Note: TEST 12: Releasing key and finishing simulation
155 -- #    Time: 20500010 ns  Iteration: 0  Instance: /keypad_tb
156 -- # ** Note: Simulation completed successfully!
157 -- #    Time: 23500010 ns  Iteration: 0  Instance: /keypad_tb
```

Listing 3: keypad_tb

4.4 Passcode

De testbench voert hier de volgende tests uit:
1. Er wordt gekeken of bij een correct wachtword het signaal wordt doorgegeven dat het wachtwoord klopt.
2. Er wordt gekeken of het systeem goed wordt gereset.
3. Er wordt gekeken of bij een fout wachtword het signaal wordt doorgegeven dat het wachtwoord niet klopt.
4. Dit is een combinatie van test 1 & 2, waarbij het systeem eerst wordt gereset, waarna het juiste wachtwoord wordt ingevoerd.

```
0 -- gen with gemini
1 -- make a testbench for it that checks 1x the right code and checks iwth
     assert that it gives correct, then reset it and try af fault code, and
     afther that test a right code again
2
3 library ieee;
4 use ieee.std_logic_1164.all;
```

```
use ieee.numeric_std.all;

entity passcode_tb is
end entity;

architecture sim of passcode_tb is

    -- Component declaration for the passcode unit under test (UUT)
    component passcode is
        port (
          clk         : in std_ulogic;
          reset       : in std_ulogic;
          hexNibble   : in std_ulogic_vector(3 downto 0);
          pressed     : in std_ulogic;
          codeCorrect : out std_ulogic;
          codeFault   : out std_ulogic;
          digitPuls   : out std_ulogic;
          codeReset   : out std_ulogic
        );
    end component;

    -- Testbench signals
    signal tb_reset       : std_ulogic := '0';
    signal tb_hexNibble   : std_ulogic_vector(3 downto 0) := "0000";
    signal tb_pressed     : std_ulogic := '0';
    signal tb_clk : std_ulogic := '0';

    signal tb_codeCorrect : std_ulogic;
    signal tb_codeFault   : std_ulogic;
    signal tb_digitPuls   : std_ulogic;
    signal tb_codeReset   : std_ulogic;

begin

    -- Instantiate the Unit Under Test (UUT)
    uut: passcode
      port map (
        clk         => tb_clk,
        reset       => tb_reset,
        hexNibble   => tb_hexNibble,
        pressed     => tb_pressed,
        codeCorrect => tb_codeCorrect,
        codeFault   => tb_codeFault,
        digitPuls   => tb_digitPuls,
        codeReset   => tb_codeReset
    );
    clk_process : process
    begin
        while true loop
            tb_clk <= '0';
            wait for 5 ns;

            tb_clk <= '1';
            wait for 5 ns;
```

```
        end loop;
    end process;

    -- Stimulus process
    stim_proc: process

        -- Helper procedure to simulate pressing a key
        procedure press_key(nibble : in std_ulogic_vector(3 downto 0)) is
        begin
            tb_hexNibble <= nibble;
            wait for 20 ns;
            tb_pressed <= '1'; -- Rising edge (registers digit)
            wait for 20 ns;
            tb_pressed <= '0'; -- Falling edge (clears digit pulse)
            wait for 20 ns;
        end procedure;

    begin
        -- Initial Reset
        report "Applying initial reset...";
        tb_reset <= '1';
        tb_pressed <= '0';
        tb_hexNibble <= "0000";
        wait for 40 ns;
        tb_reset <= '0';
        wait for 40 ns;

        ------------------------------------------------------------
        -- TEST 1: Enter Correct Code ("0001", "0001", "0001", "0001")
        ------------------------------------------------------------
        report "--- TEST 1: Entering Correct Code ---";
        press_key("0001");
        press_key("0001");
        press_key("0001");
        press_key("0001"); -- 4th digit triggers evaluation

        -- Check output with assert
        assert (tb_codeCorrect = '1')
            report "TEST 1 FAILED: codeCorrect did not assert for the
    correct code!"
            severity error;

        if tb_codeCorrect = '1' then
            report "TEST 1 PASSED: Code correct asserted successfully.";
        end if;

        wait for 40 ns;

        ------------------------------------------------------------
        -- TEST 2: Reset the system
        ------------------------------------------------------------
        report "--- TEST 2: Resetting System ---";
        tb_reset <= '1';
        wait for 40 ns;
```

```
        tb_reset <= '0';
        wait for 40 ns;

        -------------------------------------------------------------
        -- TEST 3: Enter Fault Code ("0010", "0010", "0010", "0010")
        -------------------------------------------------------------
        report "--- TEST 3: Entering Fault Code ---";
        press_key("0010");
        press_key("0010");
        press_key("0010");
        press_key("0010"); -- 4th digit triggers evaluation

        -- Check output with assert
        assert (tb_codeFault = '1')
            report "TEST 3 FAILED: codeFault did not assert for the wrong code!"
            severity error;

        if tb_codeFault = '1' then
            report "TEST 3 PASSED: Code fault asserted successfully.";
        end if;

        wait for 40 ns;

        -------------------------------------------------------------
        -- TEST 4: Reset and enter Correct Code again
        -------------------------------------------------------------
        report "--- TEST 4: Resetting and Entering Correct Code Again ---";
        tb_reset <= '1';
        wait for 40 ns;
        tb_reset <= '0';
        wait for 40 ns;

        press_key("0001");
        press_key("0001");
        press_key("0001");
        press_key("0001");

        -- Check output with assert
        assert (tb_codeCorrect = '1')
            report "TEST 4 FAILED: codeCorrect did not assert on the second attempt!"
            severity error;

        if tb_codeCorrect = '1' then
            report "TEST 4 PASSED: Second correct code asserted successfully.";
        end if;

        report "--- ALL TESTS COMPLETED ---";
        wait; -- Stop simulation
    end process;

```

```
162 end architecture;
```

Listing 4: passcode_tb

4.5 Seven Segment Driver

De testbench checkt eerst of de displays uit staan na een reset-signaal. Hierna wordt er getest of er bij elke nieuwe invoer een nieuw streepje tevoorschijn komt bij de displays. Dit begint bij HEX0 en de laatste zou HEX3 moeten zijn.

```
0  library ieee;
1  use ieee.std_logic_1164.all;
2
3  entity seven_segment_driver_tb is
4  end entity;
5
6  architecture simulation of seven_segment_driver_tb is
7
8      -- Signals connected to the DUT
9      signal reset     : std_ulogic := '0';
10     signal digitPuls : std_ulogic := '0';
11
12     signal HEX0 : std_ulogic_vector(6 downto 0);
13     signal HEX1 : std_ulogic_vector(6 downto 0);
14     signal HEX2 : std_ulogic_vector(6 downto 0);
15     signal HEX3 : std_ulogic_vector(6 downto 0);
16
17     -- Expected seven-segment values
18     constant DASH  : std_ulogic_vector(6 downto 0) := "0111111";
19     constant BLANK : std_ulogic_vector(6 downto 0) := "1111111";
20
21 begin
22
23     -- Instantiate the Device Under Test (DUT)
24     DUT: entity work.seven_segment_driver(behavioral)
25         port map (
26             reset     => reset,
27             digitPuls => digitPuls,
28             HEX0      => HEX0,
29             HEX1      => HEX1,
30             HEX2      => HEX2,
31             HEX3      => HEX3
32         );
33
34
35     -- Test process
36     stim_proc: process
37     begin
38
39         ------------------------------------------------------------------
40         -- Test 1: Reset
41         ------------------------------------------------------------------
42         reset <= '1';
```

```
        wait for 10 ns;
        reset <= '0';
        wait for 10 ns;

        assert HEX0 = BLANK and HEX1 = BLANK and
               HEX2 = BLANK and HEX3 = BLANK
            report "Test failed: Displays should be blank after reset"
            severity error;


        ------------------------------------------------------------------
        -- Test 2: First pulse
        ------------------------------------------------------------------
        digitPuls <= '1';
        wait for 10 ns;
        digitPuls <= '0';
        wait for 10 ns;

        assert HEX0 = DASH and HEX1 = BLANK and
               HEX2 = BLANK and HEX3 = BLANK
            report "Test failed: HEX0 should show DASH after first pulse"
            severity error;


        ------------------------------------------------------------------
        -- Test 3: Second pulse
        ------------------------------------------------------------------
        digitPuls <= '1';
        wait for 10 ns;
        digitPuls <= '0';
        wait for 10 ns;

        assert HEX0 = DASH and HEX1 = DASH and
               HEX2 = BLANK and HEX3 = BLANK
            report "Test failed: HEX0 and HEX1 should show DASH"
            severity error;


        ------------------------------------------------------------------
        -- Test 4: Third pulse
        ------------------------------------------------------------------
        digitPuls <= '1';
        wait for 10 ns;
        digitPuls <= '0';
        wait for 10 ns;

        assert HEX0 = DASH and HEX1 = DASH and
               HEX2 = DASH and HEX3 = BLANK
            report "Test failed: HEX0, HEX1 and HEX2 should show DASH"
            severity error;


        ------------------------------------------------------------------
        -- Test 5: Fourth pulse
```

```
        ------------------------------------------------------------------
        digitPuls <= '1';
        wait for 10 ns;
        digitPuls <= '0';
        wait for 10 ns;

        assert HEX0 = DASH and HEX1 = DASH and
               HEX2 = DASH and HEX3 = DASH
            report "Test failed: All displays should show DASH"
            severity error;


        ------------------------------------------------------------------
        -- Test 6: Extra pulse should not increase count beyond 4
        ------------------------------------------------------------------
        digitPuls <= '1';
        wait for 10 ns;
        digitPuls <= '0';
        wait for 10 ns;

        assert HEX0 = DASH and HEX1 = DASH and
               HEX2 = DASH and HEX3 = DASH
            report "Test failed: Count should not exceed 4"
            severity error;


        report "All tests completed successfully!" severity note;

        wait;
    end process;

end architecture;

-- Output:
-- # ** Note: All tests completed successfully!
-- #    Time: 120 ns  Iteration: 0  Instance: /seven_segment_driver_tb
```

Listing 5: seven_segment_driver_tb

4.6 State Machine

De testbench voert hier, voor elke transitie naar een andere staat, een test uit om te bepalen of deze daadwerkelijk gebeurt bij de bijbehorende waarden.

```
--- AI generated testbensh
--- promt: make a testbensh for this vhdl code (code added)
--- and i added the satemachine diagram
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity tb_statemachine is
end entity;
```

```
9
10 architecture simulation of tb_statemachine is
11
12     -- Input signals
13     signal codeCorrect  : std_ulogic := '0';
14     signal codeFault    : std_ulogic := '0';
15     signal codeReset    : std_ulogic := '0';
16     signal resetFromTime: std_ulogic := '0';
17     signal clk          : std_ulogic := '0';
18     signal pressed      : std_ulogic := '0';
19
20     -- Output signals
21     signal resetOut     : std_ulogic;
22     signal faultCode    : std_ulogic;
23     signal correctCode  : std_ulogic;
24     signal ledOut       : std_ulogic;
25     signal resetTimeOutClock   : std_ulogic;
26
27     constant CLK_PERIOD : time := 10 ns;
28
29 begin
30
31     -- Instantiate the Unit Under Test (UUT)
32     UUT: entity work.statemachine
33         port map (
34             codeCorrect   => codeCorrect,
35             codeFault     => codeFault,
36             codeReset     => codeReset,
37             resetFromTime => resetFromTime,
38             clk           => clk,
39             pressed       => pressed,
40
41             resetOut      => resetOut,
42             faultCode     => faultCode,
43             correctCode   => correctCode,
44             ledOut        => ledOut,
45             resetTimeOutClock    => resetTimeOutClock
46         );
47
48     -- Clock generation
49     clk_process : process
50     begin
51         while true loop
52             clk <= '0';
53             wait for CLK_PERIOD / 2;
54             clk <= '1';
55             wait for CLK_PERIOD / 2;
56         end loop;
57     end process;
58
59     -- Test process
60     stimulus : process
61
62         -- Wait for clock edge and check outputs
```

```
63          procedure check_state(
64              constant expected_resetOut    : std_ulogic;
65              constant expected_faultCode   : std_ulogic;
66              constant expected_correctCode : std_ulogic;
67              constant expected_ledOut      : std_ulogic;
68              constant expected_resetTimeOutClock  : std_ulogic;
69              constant state_name           : string
70          ) is
71          begin
72              wait until rising_edge(clk);
73              wait for 1 ns;
74
75              assert resetOut = expected_resetOut
76                  report "resetOut error in " & state_name
77                  severity error;
78
79              assert faultCode = expected_faultCode
80                  report "faultCode error in " & state_name
81                  severity error;
82
83              assert correctCode = expected_correctCode
84                  report "correctCode error in " & state_name
85                  severity error;
86
87              assert ledOut = expected_ledOut
88                  report "ledOut error in " & state_name
89                  severity error;
90
91              assert resetTimeOutClock = expected_resetTimeOutClock
92                  report "resetTimeOutClock error in " & state_name
93                  severity error;
94
95              report "Passed: " & state_name
96                  severity note;
97          end procedure;
98
99      begin
100
101         -------------------------------------------------
102         -- TEST 1: CLOSED STATE
103         -------------------------------------------------
104
105         pressed       <= '0';
106         codeCorrect   <= '0';
107         codeFault     <= '0';
108         codeReset     <= '0';
109         resetFromTime <= '0';
110
111         check_state('1', '0', '0', '1', '1', "Closed_s initial");
112
113         -------------------------------------------------
114         -- TEST 2: CLOSED -> PROGRESS
115         -------------------------------------------------
116
```

```
117         pressed <= '1';
118
119         check_state('0', '0', '0', '1', '1', "Progress_s");
120
121         -------------------------------------------------
122         -- TEST 3: PROGRESS -> PROGRESS
123         -- No correct or fault code
124         -------------------------------------------------
125
126         pressed <= '0';
127
128         check_state('0', '0', '0', '1', '1', "Progress_s hold");
129
130         -------------------------------------------------
131         -- TEST 4: PROGRESS -> CORRECT
132         -------------------------------------------------
133
134         codeCorrect <= '1';
135
136         check_state('0', '0', '1', '0', '0', "Correct_s");
137
138         -------------------------------------------------
139         -- TEST 5: CORRECT -> CORRECT
140         -- Timer has not expired
141         -------------------------------------------------
142
143         resetFromTime <= '0';
144
145         check_state('0', '0', '1', '0', '0', "Correct_s hold");
146
147         -------------------------------------------------
148         -- TEST 6: CORRECT -> CLOSED
149         -- Timer expires
150         -------------------------------------------------
151
152         codeCorrect   <= '0';
153         resetFromTime <= '1';
154
155         check_state('1', '0', '0', '1', '1', "Correct to Closed");
156
157         -------------------------------------------------
158         -- TEST 7: CLOSED -> PROGRESS
159         -------------------------------------------------
160
161         resetFromTime <= '0';
162         pressed       <= '1';
163
164         check_state('0', '0', '0', '1', '1', "Progress_s again");
165
166         -------------------------------------------------
167         -- TEST 8: PROGRESS -> FAULT
168         -------------------------------------------------
169
170         pressed   <= '0';
```

```
171         codeFault <= '1';
172
173         check_state('0', '1', '0', '1', '0', "Fault_s");
174
175         -------------------------------------------------
176         -- TEST 9: FAULT -> FAULT
177         -- Timer has not expired
178         -------------------------------------------------
179
180         codeFault <= '0';
181
182         check_state('0', '1', '0', '1', '0', "Fault_s hold");
183
184         -------------------------------------------------
185         -- TEST 10: FAULT -> CLOSED
186         -- Timer expires
187         -------------------------------------------------
188
189         resetFromTime <= '1';
190
191         check_state('1', '0', '0', '1', '1', "Fault to Closed");
192
193         -------------------------------------------------
194         -- TEST 11: CLOSED -> PROGRESS
195         -------------------------------------------------
196
197         resetFromTime <= '0';
198         pressed       <= '1';
199
200         check_state('0', '0', '0', '1', '1', "Progress_s reset test");
201
202         -------------------------------------------------
203         -- TEST 12: PROGRESS -> CLOSED
204         -- Reset button activated
205         -------------------------------------------------
206
207         pressed   <= '0';
208         codeReset <= '1';
209
210         check_state('1', '0', '0', '1', '1', "Progress to Closed");
211
212         -------------------------------------------------
213         -- TEST 13: CLOSED -> PROGRESS
214         -------------------------------------------------
215
216         codeReset <= '0';
217         pressed   <= '1';
218
219         check_state('0', '0', '0', '1', '1', "Progress_s final");
220
221         -------------------------------------------------
222         -- TEST 14: BOTH CORRECT AND FAULT
223         -- Machine should remain in Progress
224         -------------------------------------------------
```

```
225
226         pressed     <= '0';
227         codeCorrect <= '1';
228         codeFault   <= '1';
229
230         check_state('0', '0', '0', '1', '1', "Both codes activated");
231
232         ------------------------------------------------
233         -- TEST 15: CORRECT AND RESET SIMULTANEOUSLY
234         -- Reset should send machine to Closed
235         ------------------------------------------------
236
237         codeReset <= '1';
238
239         check_state('1', '0', '0', '1', '1', "Correct and Reset");
240
241         ------------------------------------------------
242         -- TEST 16: FAULT AND RESET SIMULTANEOUSLY
243         ------------------------------------------------
244
245         pressed     <= '1';
246         codeCorrect <= '0';
247         codeFault   <= '0';
248         codeReset   <= '0';
249
250         check_state('0', '0', '0', '1', '1', "Progress before reset");
251
252         pressed   <= '0';
253         codeFault <= '1';
254         codeReset <= '1';
255
256         check_state('1', '0', '0', '1', '1', "Fault and Reset");
257
258         ------------------------------------------------
259         -- END OF SIMULATION
260         ------------------------------------------------
261
262         report "ALL TESTS COMPLETED SUCCESSFULLY!"
263             severity note;
264
265         std.env.stop;
266         wait;
267
268     end process;
269
270 end architecture simulation;
271
272 -- Output:
273 -- run -all
274 -- # ** Note: Passed: Closed_s initial
275 -- #    Time: 6 ns  Iteration: 0  Instance: /tb_statemachine
276 -- # ** Note: Passed: Progress_s
277 -- #    Time: 16 ns  Iteration: 0  Instance: /tb_statemachine
278 -- # ** Note: Passed: Progress_s hold
```

```
-- #    Time: 26 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Correct_s
-- #    Time: 36 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Correct_s hold
-- #    Time: 46 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Correct to Closed
-- #    Time: 56 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Progress_s again
-- #    Time: 66 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Fault_s
-- #    Time: 76 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Fault_s hold
-- #    Time: 86 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Fault to Closed
-- #    Time: 96 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Progress_s reset test
-- #    Time: 106 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Progress to Closed
-- #    Time: 116 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Progress_s final
-- #    Time: 126 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Both codes activated
-- #    Time: 136 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Correct and Reset
-- #    Time: 146 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Progress before reset
-- #    Time: 156 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: Passed: Fault and Reset
-- #    Time: 166 ns  Iteration: 0  Instance: /tb_statemachine
-- # ** Note: ALL TESTS COMPLETED SUCCESSFULLY!
-- #    Time: 166 ns  Iteration: 0  Instance: /tb_statemachine
```

Listing 6: statemachine_tb

5 VHDL Code

De volledige code is hieronder te zien.

Dit stukje code is concurrent omdat dit alleen maar de verschillende delen met elkaar verbind

```
library ieee;
use IEEE.numeric_std.all;
use IEEE.std_logic_1164.all;

entity keypadtoplevel is
  port(
    keypadCols: in std_ulogic_vector(3 downto 0);
    clk: in std_ulogic;

    SEG0: out std_ulogic_vector(6 downto 0);
    SEG1: out std_ulogic_vector(6 downto 0);
    SEG2: out std_ulogic_vector(6 downto 0);
    SEG3: out std_ulogic_vector(6 downto 0);
    keypadRows: out std_ulogic_vector(3 downto 0);
    ledOut: out std_ulogic;
    buzzerOut: out std_ulogic
  );
end entity;

architecture behavioral of keypadtoplevel is
  component ClockDivider is
    Generic (
      counter : natural := 250000000
    );
    Port (
      reset: in std_ulogic;
      clockOut: out std_ulogic;
      clk : in std_ulogic
    );
  end component;

  component keypad is
    port(
      cols: in std_ulogic_vector(3 downto 0);
      rows: out std_ulogic_vector(3 downto 0);
      clk: in std_ulogic;

      pressed: out std_ulogic;
      keycode: out std_ulogic_vector(3 downto 0)
    );
  end component;

  component passcode is
    port (
      clk: in std_ulogic;
      reset: in std_ulogic;
      hexNibble: in std_ulogic_vector(3 downto 0);
      pressed: in std_ulogic;

```

```
49        codeCorrect: out std_ulogic;
50        codeFault: out std_ulogic;
51        digitPuls: out std_ulogic;
52        codeReset: out std_ulogic
53      );
54    end component;
55
56    component seven_segment_driver is
57      port (
58        reset: in  std_ulogic;
59        digitPuls: in  std_ulogic;
60        HEX0: out std_ulogic_vector(6 downto 0);
61        HEX1: out std_ulogic_vector(6 downto 0);
62        HEX2: out std_ulogic_vector(6 downto 0);
63        HEX3: out std_ulogic_vector(6 downto 0)
64      );
65    end component;
66
67    component statemachine is
68      port (
69        codeCorrect: in std_ulogic;
70        codeFault: in std_ulogic;
71        codeReset: in std_ulogic;
72        resetFromTime: in std_ulogic;
73        clk: in std_ulogic;
74        pressed: in std_ulogic;
75
76        resetOut: out std_ulogic;
77        faultCode: out std_ulogic;
78        correctCode: out std_ulogic;
79        ledOut: out std_ulogic;
80        resetTimeOutClock: out std_ulogic
81      );
82    end component;
83
84    component buzzer is
85      port(
86        clk: in  std_ulogic;
87        resetPuls: in  std_ulogic;
88        pressPuls: in  std_ulogic;
89        faultPuls: in  std_ulogic;
90        correctPuls: in  std_ulogic;
91        buzzer: out std_ulogic
92      );
93    end component;
94
95    -- signals
96    signal keyPressed : std_ulogic;
97    signal keyCode : std_ulogic_vector(3 downto 0);
98    signal passCodeCorrectSig : std_ulogic;
99    signal passCodeFaultSig : std_ulogic;
100   signal passCodeResetSig : std_ulogic;
101   signal digitPulsSig : std_ulogic;
102   signal resetFromTimeSig : std_ulogic;
```

```
103
104   -- output statemachine
105   signal resetComponentsOut : std_ulogic;
106   signal buzzerFault : std_ulogic;
107   signal buzzerCorrect : std_ulogic;
108   signal resetClock : std_ulogic;
109 begin
110
111   clockdividermodule: ClockDivider generic map (
112     counter => 250000000
113   ) port map (
114     reset => resetClock,
115     clockOut => resetFromTimeSig,
116     clk => clk
117   );
118
119   statemachinemodule : statemachine port map (
120     codeCorrect => passCodeCorrectSig,
121     codeFault => passCodeFaultSig,
122     codeReset => passCodeResetSig,
123     pressed => keyPressed,
124     clk => clk,
125     resetFromTime => resetFromTimeSig,
126
127     resetOut => resetComponentsOut,
128     faultCode => buzzerFault,
129     correctCode => buzzerCorrect,
130     ledOut => ledOut,
131     resetTimeOutClock => resetClock
132   );
133
134   passcodemodule : passcode port map (
135     clk => clk,
136     reset => resetComponentsOut,
137     hexNibble => keyCode,
138     pressed => keyPressed,
139
140     codeCorrect => passCodeCorrectSig,
141     codeFault => passCodeFaultSig,
142     digitPuls => digitPulsSig,
143     codeReset => passCodeResetSig
144   );
145
146   sevensegmodule : seven_segment_driver port map (
147     reset => resetComponentsOut,
148     digitPuls => digitPulsSig,
149     HEX0 => SEG0,
150     HEX1 => SEG1,
151     HEX2 => SEG2,
152     HEX3 => SEG3
153   );
154
155   keypadmodule : keypad port map (
156     cols => keypadCols,
```

```
157     rows => keypadRows,
158     clk => clk,
159     pressed => keyPressed,
160     keycode => keyCode
161   );
162
163   buzzermodule : buzzer port map (
164     clk => clk,
165     resetPuls => resetComponentsOut,
166     pressPuls => digitPulsSig,
167     faultPuls => buzzerFault,
168     correctPuls => buzzerCorrect,
169     buzzer => buzzerOut
170   );
171
172 end architecture;
```

Listing 7: keypadtoplevel.vhd

5.1 Keypad

Het volgende stuk code is voor de keypad, er is besloten dat deze alleen een puls stuurt wanneer er een key is ingedrukt en daarnaast altijd de keycode, dit is omdat alle andere componenten dan geen rekening hoeven te houden met hoe de keypad werkt. Deze code is sequential omdat deze op een frequentie van 100hz per rij werkt omdat dat meer dan genoeg is.

```
library ieee;
use ieee.numeric_std.all;
use ieee.std_logic_1164.all;

entity keypad is
  port(
    cols     : in  std_ulogic_vector(3 downto 0);
    rows     : out std_ulogic_vector(3 downto 0);
    clk      : in  std_ulogic;

    pressed  : out std_ulogic;
    keycode  : out std_ulogic_vector(3 downto 0)
  );
end entity;

architecture behavioral of keypad is

  component ClockDivider is
    generic (
      counter : natural := 25000000
    );
    port (
      reset    : in  std_ulogic;
      clockOut : out std_ulogic;
      clk      : in  std_ulogic
    );
  end component;

  signal currentRow : integer range 0 to 3 := 0;

  signal scanRow  : std_ulogic;
  signal reset    : std_ulogic := '0';
  signal outClock : std_ulogic;

  signal lastKey    : std_ulogic_vector(3 downto 0) := (others => '0');
  signal currentKey : std_ulogic_vector(3 downto 0) := (others => '0');

  signal detected   : std_ulogic := '0';
  signal keyReleased : std_ulogic := '1';

begin

  clockInst : ClockDivider
    generic map (
      counter => 125000
    )
    port map (
      reset    => reset,
```

```
48        clockOut => scanRow,
49        clk      => clk
50      );
51
52    clockInst2 : ClockDivider
53      generic map (
54        counter => 5000
55      )
56      port map (
57        reset    => reset,
58        clockOut => outClock,
59        clk      => clk
60      );
61
62    ------------------------------------------------
63    -- Keypad scanning process
64    ------------------------------------------------
65
66    process(scanRow)
67
68      variable vKey      : std_ulogic_vector(3 downto 0);
69      variable vDetected : std_ulogic;
70
71    begin
72
73      if rising_edge(scanRow) then
74        vKey := currentKey;
75        vDetected := detected;
76
77        if currentRow = 0 then
78          vDetected := '0';
79          vKey := "1111";
80        end if;
81
82        case currentRow is
83          when 0 => rows <= "1110";
84          when 1 => rows <= "1101";
85          when 2 => rows <= "1011";
86          when 3 => rows <= "0111";
87          when others => rows <= "1111";
88        end case;
89
90        case cols is
91          when "0111" =>
92            case currentRow is
93              when 0 =>
94                vKey := "0001";
95                vDetected := '1';
96
97              when 1 =>
98                vKey := "1110";
99                vDetected := '1';
100
101             when 2 =>
```

```
102               vKey := "0111";
103               vDetected := '1';
104
105             when 3 =>
106               vKey := "0100";
107               vDetected := '1';
108
109             when others => null;
110           end case;
111
112         when "1011" =>
113           case currentRow is
114             when 0 =>
115               vKey := "0010";
116               vDetected := '1';
117
118             when 1 =>
119               vKey := "0000";
120               vDetected := '1';
121
122             when 2 =>
123               vKey := "1000";
124               vDetected := '1';
125
126             when 3 =>
127               vKey := "0101";
128               vDetected := '1';
129
130             when others => null;
131           end case;
132
133         when "1101" =>
134           case currentRow is
135             when 0 =>
136               vKey := "0011";
137               vDetected := '1';
138
139             when 2 =>
140               vKey := "1001";
141               vDetected := '1';
142
143             when 3 =>
144               vKey := "0110";
145               vDetected := '1';
146
147             when others => null;
148           end case;
149         when others => null;
150       end case;
151
152       currentKey <= vKey;
153       detected <= vDetected;
154
155       if currentRow = 3 then
```

```
156         currentRow <= 0;
157         if vDetected = '1' then
158           if keyReleased = '1' or lastKey /= vKey then
159             pressed <= '1';
160             lastKey <= vKey;
161             keycode <= vKey;
162             keyReleased <= '0';
163           else
164             pressed <= '0';
165           end if;
166         else
167           if cols = "1111" then
168             keyReleased <= '1';
169           end if;
170           pressed <= '0';
171         end if;
172       else
173         currentRow <= currentRow + 1;
174       end if;
175     end if;
176   end process;
177
178 end architecture;
```

Listing 8: keypad.vhd

5.2 Passcode

De passcode reageert alleen maar op pulses van de de module van de keypad, Deze module wordt gereset in de idle state van de statemachine. De code is opgeslagen in een 16 bit getal, 4 bits per cijfer. De inputs worden aan elkaar vastgeknoopt door and operatoren, waardoor de code 16 bits wordt. Deze wordt dan gecheckt met de juiste code. Als de code juist is zal het signaal voor code juist worden gestuurt, is die fout zal het signaal voor code fout gestuurt worden. Als deze module '*' (1110) krijgt, zal die de reset output geven. Dit reset het systeem waardoor de code gereset wordt. Deze module stuurt ook signalen door naar de sevensegment driver en de buzzer om te laten weten dat er een valid key is ingedrukt. Deze code is sequential omdat deze werkt met een klok.

```
library ieee;
use IEEE.numeric_std.all;
use IEEE.std_logic_1164.all;

entity passcode is
  port (
    clk: in std_ulogic;
    reset: in std_ulogic;
    hexNibble: in std_ulogic_vector(3 downto 0);
    pressed: in std_ulogic;

    codeCorrect: out std_ulogic;
    codeFault: out std_ulogic;
    digitPuls: out std_ulogic;
    codeReset: out std_ulogic
  );
end entity;

architecture behavioral of passcode is
  signal count        : integer range 0 to 4 := 0;

  signal inputdig0    : std_ulogic_vector(3 downto 0) := "0000";
  signal inputdig1    : std_ulogic_vector(3 downto 0) := "0000";
  signal inputdig2    : std_ulogic_vector(3 downto 0) := "0000";

  constant correctCode : std_ulogic_vector(15 downto 0) :=
   "0001001010010011";

  -- current and last signal from "pressed"
  signal pressedLast   : std_ulogic := '0';
  signal pressedCurrent : std_ulogic := '0';
begin

  -- rising and falling edge detection for 'pressed' signal
  process(clk, reset)
  begin
    if reset = '1' then
      pressedLast   <= '0';
      pressedCurrent <= '0';
    elsif rising_edge(clk) then
      pressedLast   <= pressed;
      pressedCurrent <= pressed and not pressedLast; -- Single-cycle pulse
    on button press
    end if;
  end process;
```

```
43
44    process(clk, reset)
45    begin
46      if reset = '1' then
47        inputdig0   <= "0000";
48        inputdig1   <= "0000";
49        inputdig2   <= "0000";
50        codeReset   <= '0';
51        codeCorrect <= '0';
52        codeFault   <= '0';
53        digitPuls   <= '0';
54        count       <= 0;
55      elsif rising_edge(clk) then
56        codeReset   <= '0';
57        codeCorrect <= '0';
58        codeFault   <= '0';
59        digitPuls   <= '0';
60
61        if pressedCurrent = '1' then
62          if hexNibble = "1110" then
63            codeReset <= '1';
64            count     <= 0;
65          else
66            digitPuls <= '1';
67            case count is
68              when 0 =>
69                inputdig0 <= hexNibble;
70                count     <= 1;
71              when 1 =>
72                inputdig1 <= hexNibble;
73                count     <= 2;
74              when 2 =>
75                inputdig2 <= hexNibble;
76                count     <= 3;
77              when 3 =>
78                count     <= 4;
79                if (inputdig0 & inputdig1 & inputdig2 & hexNibble) =
    correctCode then
80                  codeCorrect <= '1';
81                else
82                  codeFault <= '1';
83                end if;
84
85              when others =>
86                codeFault <= '1';
87                count     <= 0;
88            end case;
89          end if;
90        end if;
91      end if;
92    end process;
93
94  end architecture;
```

Listing 9: passcode.vhd

5.3 Buzzer

De buzzer is een module voor de externe buzzer die aangesloten is op het ontwerp. Deze zit op de GPIO_0_D0 pin. De buzzer heeft een statemachine, die te zien is in het ontwerp hoofdstuk. Deze code zorgt voor dat er piepjes komen als er een code wordt intoetst. Deze module heeft een klok input, en is sequentieel, en heeft vier ander inputs: reset, press, fault en correct. Deze spelen hun geluidjes af wanneer de pin hoog wordt. De press key bij iedere toets, de reset key als het systeem gereset wordt, de correct key als de code juist is en de fault key als de code fout is.

```
0   /* Buzzer-component: Een buzzer gaat aan wanneer er op de keypad wordt
      gedrukt, */
1  /*  wanneer de code goed/fout is, en waneer er op de reset-knop wordt
      gedrukt.  */
2 /*   Hiervoor zijn drie verschillende vormen van geluid bedacht.
          */
3 library ieee;
4 use ieee.std_logic_1164.all;
5
6 entity buzzer is
7     generic (
8         clkFreq: natural := 50000000
9     );
10    port (
11        clk: in std_ulogic;
12        pressPuls: in std_ulogic;
13        faultPuls: in std_ulogic;
14        correctPuls: in std_ulogic;
15        resetPuls: in std_ulogic;
16        buzzer: out std_ulogic -- Geluid is te horen wanneer 'buzzer' hoog
      is.
17    );
18 end entity;
19
20 architecture behavorial of buzzer is
21    constant SHORTBUZZ: natural := clkFreq / 8; -- Korte piep: buzzer gaat
      aan voor 125 milliseconden.
22    constant BUZZGAP: natural := (clkFreq / 8) * 3; -- Tussen twee korte
     piepjes is er een pauze van 375 milliseconden.
23    constant LONGBUZZ: natural := clkFreq / 2;-- Lange piep: buzzer gaat
     aan voor 500 milliseconden.
24
25    type state_type is (idle, buzz1, gap, buzz2);
26    signal state : state_type := idle;
27    signal doubleBuzz: std_ulogic := '0';
28    signal timer: natural range 0 to BUZZGAP + LONGBUZZ := 0;
29
30    signal s1, s2, prev: std_ulogic_vector(3 downto 0) := (others => '0');
31 begin
32    process (clk)
33        variable rise: std_ulogic_vector(3 downto 0);
34    begin
35        if rising_edge(clk) then
36            s1 <= correctPuls & faultPuls & resetPuls & pressPuls;
37            s2 <= s1;
38            prev <= s2;
```

```
39              rise := s2 and not prev;
40
41          if rise(3) = '1' then -- code correct: 2 korte piepjes
42              state <= buzz1;
43              timer <= SHORTBUZZ - 1;
44              doubleBuzz <= '1';
45          elsif rise(2) = '1' then -- code fout: 1 lange piep
46              state <= buzz1;
47              timer <= LONGBUZZ - 1;
48              doubleBuzz <= '0';
49          elsif rise(1) = '1' then -- reset code: 1 lange piep
50              state <= buzz1;
51              timer <= LONGBUZZ - 1;
52              doubleBuzz <= '0';
53          elsif rise(0) = '1' then -- knop ingedrukt: 1 korte piep
54              state <= buzz1;
55              timer <= SHORTBUZZ - 1;
56              doubleBuzz <= '0';
57          else
58              case state is
59                  when idle =>
60                      null;
61                  when buzz1 =>
62                      if timer = 0 then
63                          if doubleBuzz = '1' then
64                              state <= gap;
65                              timer <= BUZZGAP - 1;
66                          else
67                              state <= idle;
68                          end if;
69                      else
70                          timer <= timer - 1;
71                      end if;
72                  when gap =>
73                      if timer = 0 then
74                          state <= buzz2;
75                          timer <= SHORTBUZZ - 1;
76                      else
77                          timer <= timer - 1;
78                      end if;
79                  when buzz2 =>
80                      if timer = 0 then
81                          state <= idle;
82                      else
83                          timer <= timer - 1;
84                      end if;
85                  end case;
86              end if;
87          end if;
88      end process;
89
90      buzzer <= '1' when (state = buzz1 or state = buzz2) else '0';
91  end architecture;
```

Listing 10: buzzer.vhd

5.4 Sevensegment driver

Deze module zet streepjes op het scherm wanneer deze een puls krijgt van de passcode module. Deze wordt gereset door de statemachine door middel van de rest pin. Deze module telt het aantal pulsen dat die krijgt en laat een dash zien op de segment display die er bij hoort. Deze module telt het aantal pulsen en bij vier gaat die niet meer hoger en wordt pas nul als die gerest wordt. Deze module is ook concurrent omdat deze niet geklokt wordt, hij hoogt alleen een getal op, en als het getal hoog genoeg is laat hij een dash zien.

```
0  library ieee;
1  use ieee.std_logic_1164.all;
2
3  entity seven_segment_driver is
4      port (
5          reset: in  std_ulogic;
6          digitPuls: in  std_ulogic;
7          HEX0: out std_ulogic_vector(6 downto 0);
8          HEX1: out std_ulogic_vector(6 downto 0);
9          HEX2: out std_ulogic_vector(6 downto 0);
10         HEX3: out std_ulogic_vector(6 downto 0)
11     );
12 end entity;
13
14 architecture behavioral of seven_segment_driver is
15     constant DASH: std_ulogic_vector(6 downto 0) := "0111111";
16     constant BLANK: std_ulogic_vector(6 downto 0) := "1111111";
17
18     signal count: natural range 0 to 4 := 0;
19 begin
20     process (digitPuls, reset)
21     begin
22         if reset = '1' then
23             count <= 0;
24         elsif rising_edge(digitPuls) then
25             if count < 4 then
26                 count <= count + 1;
27             end if;
28         end if;
29     end process;
30
31     HEX0 <= DASH when count >= 1 else BLANK;
32     HEX1 <= DASH when count >= 2 else BLANK;
33     HEX2 <= DASH when count >= 3 else BLANK;
34     HEX3 <= DASH when count >= 4 else BLANK;
35 end architecture;
```

Listing 11: seven_segment_driver

5.5 Statemachine

De statemachine heeft 4 states, Closed, Progress, Correct en fault. Hoe de states werken is te zien in het statediagram. De statemachine is sequentieel.

```
library ieee;
use IEEE.numeric_std.all;
use IEEE.std_logic_1164.all;

entity statemachine is
  port (
    codeCorrect: in std_ulogic;
    codeFault: in std_ulogic;
    codeReset: in std_ulogic;
    resetFromTime: in std_ulogic;
    clk: in std_ulogic;
    pressed: in std_ulogic;

    resetOut: out std_ulogic;
    faultCode: out std_ulogic;
    correctCode: out std_ulogic;
    ledOut: out std_ulogic;
    resetTimeOutClock: out std_ulogic
  );
end entity;

architecture structural of statemachine is
  type stateType is (Closed_s, Progress_s, Correct_s, Fault_s);
  signal currentState, nextState: stateType;
begin

  flipflops : process (clk)
  begin
    if rising_edge(clk) then
      currentState <= nextState;
    end if;
  end process;

  process(clk, codeCorrect, codeFault, codeReset, pressed, resetFromTime)
  begin
    case currentState is
      when Closed_s =>
        if pressed = '1' then
          nextState <= Progress_s;
        else
          nextState <= Closed_s;
        end if;

      when Progress_s =>
        if codeCorrect = '1' and not (codeFault = '1' or codeReset = '1')
   then
          nextState <= Correct_s;
        elsif codeFault = '1' and not (codeCorrect = '1' or codeReset =
   '1') then
          nextState <= Fault_s;
```

```
48          elsif codeReset = '1' then
49            nextState <= Closed_s;
50          else
51            nextState <= Progress_s;
52          end if;
53
54        when Correct_s =>
55          if resetFromTime = '1' then
56            nextState <= Closed_s;
57          else
58            nextState <= Correct_s;
59          end if;
60
61        when Fault_s =>
62          if resetFromTime = '1' then
63            nextState <= Closed_s;
64          else
65            nextState <= Fault_s;
66          end if;
67      end case;
68    end process;
69
70    process(currentState)
71    begin
72      case currentState is
73        when Closed_s =>
74          resetOut <= '1';
75          faultCode <= '0';
76          correctCode <= '0';
77          ledOut <= '1';
78          resetTimeOutClock <= '1';
79
80        when Progress_s =>
81          resetOut <= '0';
82          faultCode <= '0';
83          correctCode <= '0';
84          ledOut <= '1';
85          resetTimeOutClock <= '1';
86
87          when Correct_s =>
88          resetOut <= '0';
89          faultCode <= '0';
90          correctCode <= '1';
91          ledOut <= '0';
92          resetTimeOutClock <= '0';
93
94          when Fault_s =>
95          resetOut <= '0';
96          faultCode <= '1';
97          correctCode <= '0';
98          ledOut <= '1';
99          resetTimeOutClock <= '0';
100
101     end case;
```

```
102   end process;
103 end architecture structural;
```

Listing 12: statemachine.vhd

5.6 Clockdivider

De clockdivider die bij deze opdracht is gebruikt is dezelfde als die gebruikt is bij de weekelijkse opgaven.

```
0  library ieee;
1  use IEEE.std_logic_1164.all;
2  use IEEE.numeric_std.all;
3
4  entity ClockDivider is
5    generic (
6      counter: natural := 25000000
7    );
8    port (
9      reset: in std_ulogic;
10     clockOut: out std_ulogic;
11     clk : in std_ulogic
12   );
13 end entity;
14
15 architecture behavioural of ClockDivider is
16
17   signal clockSig : std_ulogic := '0';
18   signal count    : natural := 0;
19
20 begin
21   process(clk, reset)
22   begin
23     if(reset = '1') then
24       count <= 0;
25       clockSig <= '0';
26     elsif(rising_edge(clk)) then
27       if (count = counter - 1) then
28         count <= 0;
29         clockSig <= not clockSig;
30       else
31           count <= count + 1;
32       end if;
33     end if;
34   end process;
35   clockOut <= clockSig;
36 end architecture;
```

Listing 13: ClockDivider.vhd